

EXPERIMENT 24

Create a CI/CD Pipeline Using GitHub Actions

1. Aim

To create a CI/CD pipeline for a containerized Flask application using GitHub Actions, automating testing, Docker image building, pushing to Docker Hub, and cloud deployment.

2. Requirements

GitHub repository, Flask application, Docker, Docker Hub account, GitHub Actions, and a cloud platform such as Vercel/Heroku/AWS.

3. GitHub Actions Workflow

Create `.github/workflows/ci-cd.yml` in the repository:

```
name: Flask CI/CD

on:
  push:
    branches: [ main ]

jobs:
  build-test-push:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Test application
        run: pytest

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Build Docker image
        run: docker build -t ${ secrets.DOCKERHUB_USERNAME }/flask-app:latest .

      - name: Push Docker image
        run: docker push ${ secrets.DOCKERHUB_USERNAME }/flask-app:latest
```

4. GitHub Secrets

Add `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` under GitHub Repository → Settings → Secrets and variables → Actions.

5. Deployment

Connect the GitHub repository to the selected cloud platform and configure it to deploy the application after the workflow succeeds. The deployed application can then be accessed using the platform-generated deployment URL.

6. Testing / Deployment Screenshot

The supplied screenshot shows the project deployment dashboard and the Production Checklist. It provides evidence of the deployment environment and deployment configuration.

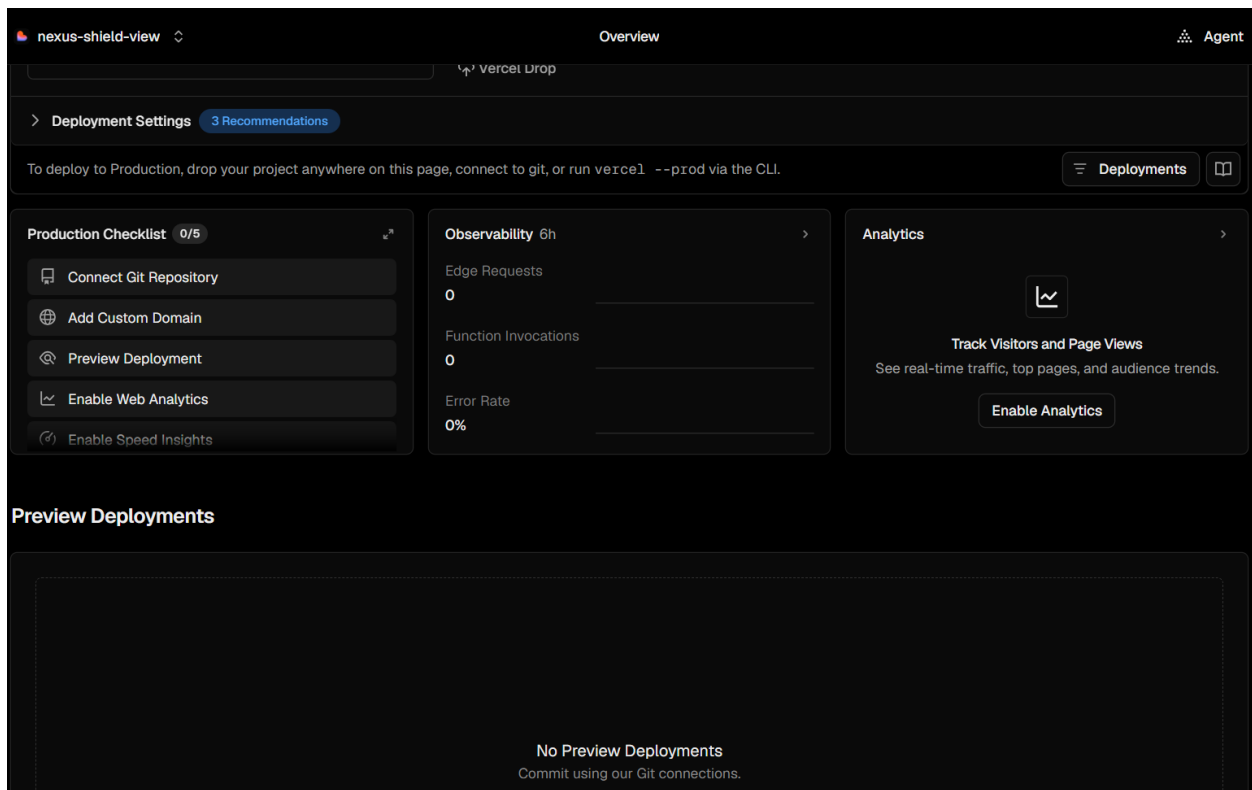


Figure 1: Cloud deployment dashboard for the Flask application.

7. Result

A CI/CD workflow was defined to automatically check out the Flask project, install dependencies, run tests, build a Docker image, and push the image to Docker Hub. The application deployment environment was also configured for cloud deployment.

8. Conclusion

GitHub Actions automates the software delivery process, reducing manual steps and enabling consistent testing, Docker image creation, and deployment.